Tutorial 3 - ER Modeling

Tutorial Objectives:

Create an ERD (using J Developer/Visio) that can be implemented for a medical clinic for the given business rules.

Tutorial Tasks:

A. Create an ERD for a government-funded medical clinic based on the given business rules:

i. A patient can make many appointments with one or more doctors in the clinic, and a doctor can accept appointments with many patients. However, each appointment is made with only one doctor and one patient.
ii. Emergency cases do not require an appointment. However, for appointment management purposes, an emergency is entered in the appointment book as "unscheduled."
iii. If kept, an appointment yields a visit with the doctor specified in the appointment. The visit yields a diagnosis and, when appropriate, treatment.
iv. With each visit, the patient's records are updated to provide a medical history.
v. Each patient visit creates a bill. Each patient visit is billed by one doctor, and each doctor can bill many patients.
vi. Each bill must be paid. However, a bill may be paid in many instalments, and a payment covers only one bill.

Your ERD should include the required entities, relevant attributes (also whether required or not), connectivity, etc. The diagram should also include the cardinalities, association names and should flag the PKs, FKs, and any derived attributes.

State your assumptions in a concise manner but at the same time do not try to alter / increase the scope based on your knowledge of some medical clinic.

Note: As the focus is on getting the modelling right, may ***first hand draw your diagram and understand conceptually*** before you create this using J Developer/Visio, etc.

The above is slightly ***adapted*** version problem 8, Ch4, 157 ed 13.

Some Notes/Tutorial Guidance/Discussion points (only when you are ready – may be use for revisiting your initial work also do not have to follow all):

- Before attempting would be better to go thru/revise the chapter 4 and relevant lecture materials. The textbook exercises are used in tutorials as they are not only great exercises, but authors would have done the mapping to ensure that these are very much doable from the given learning materials. Do pay attention to the book examples. To be able to read and understand a diagram is also a necessary skill that aids you to create your own diagram(s).
- Try first to look for things/concepts (entities) that are important in the given description/domain.

- Entities are usually proper nouns (many times they keep appearing more than once in discussion, document(s) or given text – e.g., Patient, Appointment, etc). They should also be Singular.
- Better to have more (entities) at an earlier stage rather than excluding (we can have more candidate classes/entities – e.g., emergency). You do not want to leave anything that may be important.
- Use the language of domain to describe these (e.g., Bill vs Invoice).
- Clarify the ambiguities; in our learning context state these as assumptions (should be reasonable).
- Draw entities as boxes (better to follow good naming conventions, style and consistency is very important from readability point of view). After all diagrams are mainly for the communication/discussion.
- You should be able to come up with 5 or more entities which is a great start (do not worry if you are unsure about whether some concept should be an entity or not, list that for now with a question mark/some prompt for a later review).
- Then start adding some attributes (again usually nouns, use good and consistent naming conventions) and relationships (usually verbs, you can use phrases as well) to your diagram. There can only be 1:M (usually) and 1:1 relationship (being relational). Do not worry about cardinality initially. You can later fine-tune your model and add these.
- Flag the PKs and FKs (important constraints in relational).

  Example text where some students got confused (whether Emergency should be a class or not):

  "Emergency cases do not require an appointment. However, for appointment management purposes, an emergency is entered in the appointment book as "unscheduled."

  How will your model capture this? Is Emergency an entity (candidate entity/class at this stage)?

  If this is an entity, what would be the attributes of this entity. At times doing this exercise also helps in deciding whether a concept/thing is entity or not. Sometimes, thinking about / adding some sample data* also helps. If you cannot get any attributes / data values to these, consider the option of this being an attribute; may be that can do the job (means your model can capture the design requirements). If we agree to consider this an attribute what entity will this belong to? What would be a good name for the attribute to capture this?

  We in class agreed on this be an attribute to be called Appt_scheduled in the Appointment class (could be named is_scheduled – indicating Boolean type with values – Yes, No; 1,0; True/False; etc) and followed the convention of prefixing with the table name.

Based on the required (i to vi) business rules you may get the following entities/similar entities:

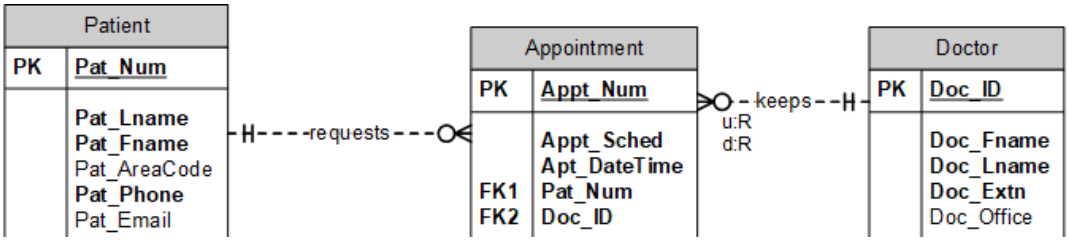
Patient, Appointment, Doctor, Visit, Bill, Patient_Payment, Visit_Detail

Note: Once again, the number of entities and relationships can be different (based on your design assumptions and/or interpretation/understanding of your requirements). The names can also vary even if the concept/entity is similar.

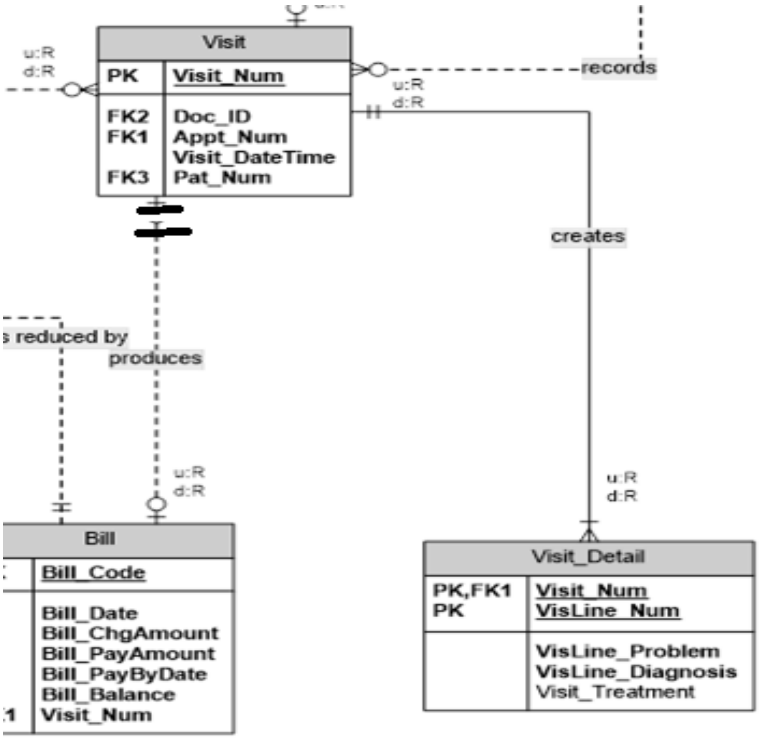
You will need to elaborate your design by adding attributes, relationships (along with names), cardinalities, flagging keys, etc.

Few screen shots of one of the possible answers.

Here is the subset of ERD that would be a starting point for most.



Here is another subset of the ERD which may warrant some discussion. Again, you might have modelled differently and that may be fine as well.



This design gives us opportunity to discuss different concepts and Task B/Task C:

- PK- single attribute (Bill_Code) vs composite (Visit_Detail); Could there be an alternative to using a composite PK?
- 1:1 (Visit and Bill) And 1:M relationships (Visit and Visit_Detail)
- Many concepts/Questions that are part of the tutorial Task B (weak or strong entities, weak and strong relationships, composite keys, etc).
- Derived attribute (e.g., Bill_Balance - Charged amount minus paid amount; is usually preceded by a forward slash "/")
- How do you make decision about the FK placement in 1:1 relationship? Why is Visit_Num a FK in Bill and why not the other way around.
- Task C, what constraint will you use? Which Bill attribute is affected by this constraint?
- Each Visit must produce (1 or more) Visit_Detail. For example, Patient Mary reports these problems/symptoms – severe headache, irregular heartbeat, runny nose, and malaise to her doctor. Her doctor might record two diagnoses (means two visit_line entries to her single visit) for her – High BP (because of severe headache, irregular heartbeat problem) and common cold (because of runny nose and malaise problem). Treatment attribute could be some BP medication and no medication for common cold (just rest, assuming not counted as treatment as there is no medicine).
- Vist_DateTime attribute captures the actual visit date and time (this could be different to the appointment date and time – they are usually running late, or no shows, or because of emergencies).
- Likewise, one could make an appoint with a preferred doctor but occasionally is seen (recorded) by a different doctor. (See if your model supports that or how would you alter your design to incorporate this change).
- Which other entity (or more than one) is related to Bill? (Both patient and patient_payment or just with Patient_payment which is further related to Patient).

Design can really vary based on the requirements/assumptions so do not worry too much about variations. This discussion is about raising the issues that may need to be considered. Always review your design (ideally with a different person, or in a different session when doing solo work) and state the assumptions (especially where you got stuck because of different possible situations/interpretation, ambiguities). Distance students can discuss such tutorials within their assignment groups as well.

Existence dependent / Existence independent entities

- **Patient and Doctor** are existence-independent entities (Why?)
- **Bill** is existence-dependent on **Visit** (Why?)
- **… (based on your design)**

Relationship degree - unary, binary, or ternary

- Usually binary, but at times there could be others as well.

Others, please attempt yourself (some discussion is already done earlier)

***Appendix:**
Some **sample data** for some of values for Bill and Pat(ient)Payment tables. There is also a **derived** attribute BillBal(ance), the values for which will change (so same record will change for that value) as payments are made by the patient (these are recoded in PatPayment table).

Bill

BillCode	BillChgAmount	BillBal	BillDate	BillPayByDate
999	100	100	2/07/2022	2/08/2022
999	100	60	2/07/2022	2/08/2022
999	100	0	2/07/2022	2/08/2022

PatPayment

PatPayNum	BillCode	PatPayDate	PatPayType	PatPayAmount	PatNum
1001	999	4/07/2022	Cash	40	1
2000	999	20/07/2022	EftPos	60	1